MODULE 1

C Fundamentals - Character Set, Constants, Identifiers, Keywords, Basic Data types, Variables, Operators and its precedence, Bit-wise operators, Expressions; Statements - Input and Output statements; Structure of a C program; Simple programs.

Control Statements - if, if-else, nested if, switch, while, do-while, for, break & continue, nested loops.

Character Set

- As every language contains a set of characters used to construct words, statements, etc. C language also has a set of characters which include ***alphabets, digits***, and ***special symbols***.
- The characters in C are grouped into the following categories
  - Letters
  - Digits
  - Special characters
  - White spaces

Letters

C language supports all the alphabets from the English language. Lower and upper case letters together support 52 alphabets.

Lower case letters - a to z

Upper case letters - A to Z

Digits

C language supports 10 digits which are used to construct numerical values in C language.

Digits - 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

Special Characters

, comma
. period
; semicolon
: colon
? question mark
' apostrophe
" quotation mark
! exclamation mark
| vertical bar
/ slash
\ backslash
~ tilde
_ under score
$ dollar sign
% percent sign
& ampersand
^ caret
* asterisk
– minus sign
+ plus sign
< opening angle bracket
(or less than sign)
> closing angle bracket
(or greater than sign)
(left parenthesis
) right parenthesis
[left bracket
] right bracket
{ left brace
} right brace
number sign

White Spaces

Blank space

Horizontal tab

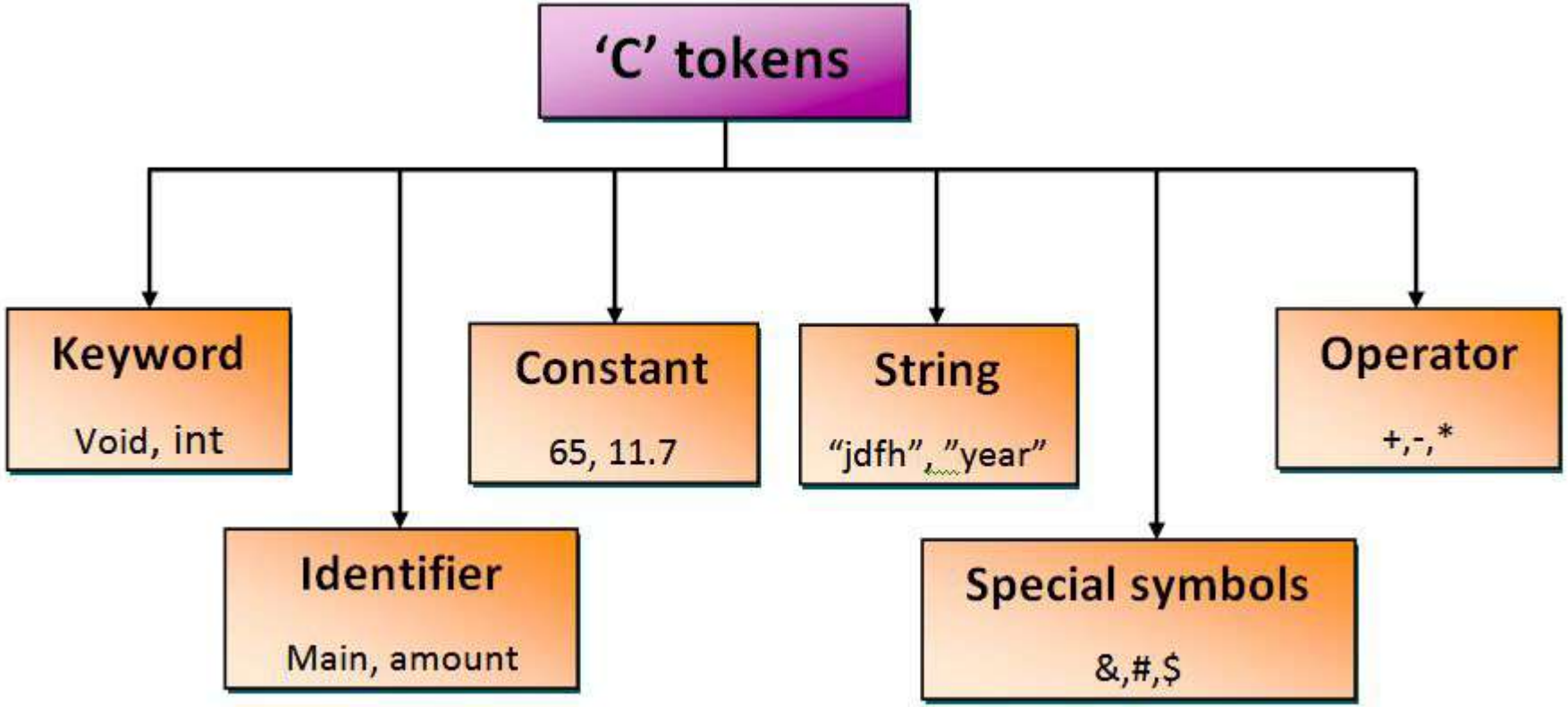
Carriage return

New line

Form feed

C Tokens

- Every C program is a collection of instructions and every instruction is a collection of some individual units.
- Every smallest individual unit of a c program is called token.
- Every instruction in a c program is a collection of tokens.
- Tokens are used to construct c programs and they are said to the basic building blocks of a c program.
- C has 6 types of tokens


'C' tokens
Keyword
Void, int
Identifier
Main, amount
Constant
65, 11.7
String
"jdfh", "year"
Special symbols
&,#,$
Operator
+,-,*

Keywords and Identifiers

- Every C word is classified as either a keyword or an identifier
- All keywords have fixed meanings and these meanings cannot be changed.
- Keywords serve as basic building blocks for program statements.
- All keywords must be written in lowercase.
- There are total of 32 keywords in C:

Keywords in C Language

Keywords in C Language			
auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
continue	for	signed	void
do	if	static	while
default	goto	sizeof	volatile
const	float	short	unsigned

Identifiers

- Identifiers refers to the names of variables,functions and arrays.
- These are user defined names and consist of sequence of letters and digits ,with a letter as a first character.
- Both upper and lower case letters are permitted,although lowercase letters are commonly used.
- The underscore character is also permitted in identifiers.It is usually used as a link between two words in long identifiers

Rules for identifiers

- A valid identifier can have letters (both uppercase and lowercase letters), digits and underscores.
- The first letter of an identifier should be either a letter or an underscore. However, it is discouraged to start an identifier name with an underscore.
- There is no rule on length of an identifier. However, the first 31 characters of identifiers are discriminated by the compiler.
- Identifiers are case sensitive ie; X is different from x.
- Keywords should not be used as identifiers.

The following are valid identifiers

x y12 sum_1 _temperature names area tax_rate TABLE

The following names are not valid identifiers.

2csbmec The first character must be a letter.

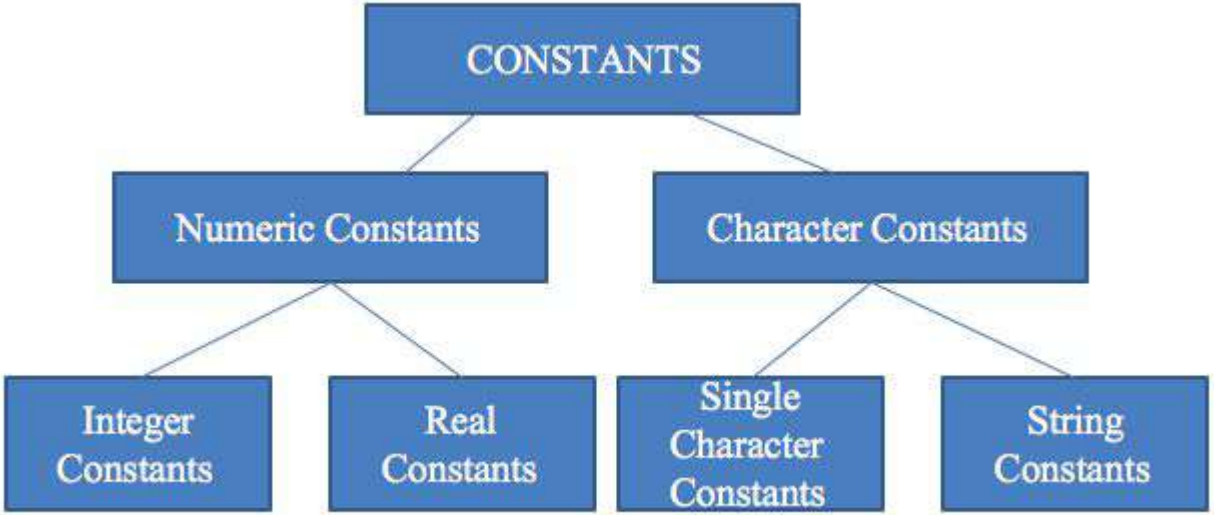
mec#cs2b Illegal character #.

order-no Illegal character (-).

mec cs2b Illegal character (blank space)

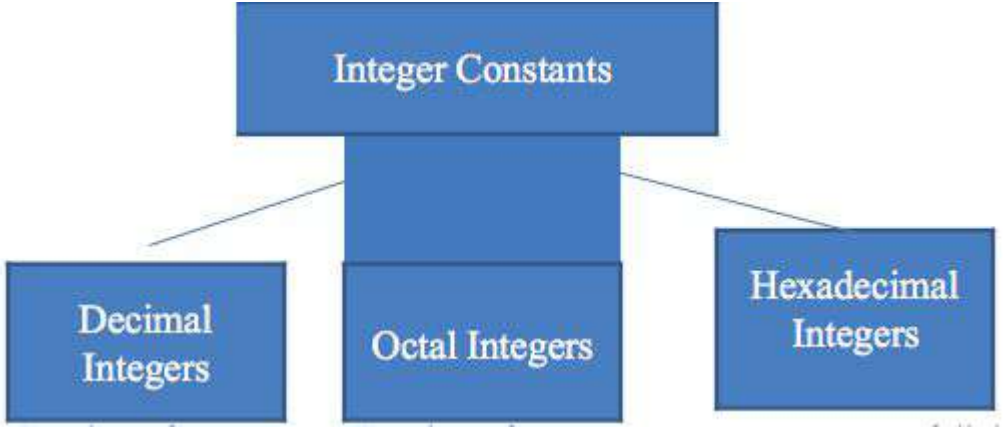
CONSTANTS

- Fixed values that do not change during the execution of a program.



Integer Constants

An integer constant refers to a sequence of digits.



- Decimal integer constants consists of a set of digits, 0 through 9, preceded by an optional – or + sign.

  Example: 123,-123,0, +73

- An octal integer constant consists of any combination of digits from 0 to 7,with a leading 0.

  Example: 037, 0, 0437

- A sequence of digits preceded by 0x or 0X is considered as hexadecimal integer constant .They may include Alphabet A through F.The letter A through F represents numbers 10 through 15.

  Example: 0X2, ox9F, oxABCD

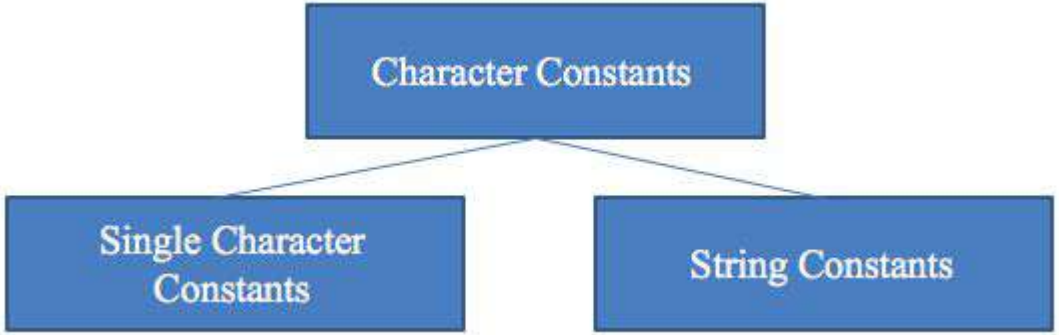
Real Constants

- These numbers are shown in decimal notation,having a whole number followed by a decimal point and the fractional part.
- A real number may expressed in ***exponential (or scientific)***notation.

  mantissa e exponent
- The mantissa is either a real number expressed in decimal notation or integer.
- The exponent is an integer number with an optional + or – sign.
- The letter e can be written in either uppercase or lowercase.
- Embedded white space is not allowed.
  - Valid real numbers: 444.643f,321.55F,2.5e+05,374e-04
  - Invalid real numbers: 1.5E+0.5,$25,ox7B,7.5 e -0.5,1,00,00.00

- Floating-point constants are normally represented as double-precision quantities in C.
- Hence, each floating-point constant will typically occupy 2 words (8 bytes) of memory.
- Some versions of C permit the specification of a “single-precision,” floating-point constant, by appending the letter F (in either upper- or lowercase) to the end of the constant (e.g., 3E5F).
- Similarly, some versions of C permit the specification of a “long” floating-point constant, by appending the letter L (upper- or lowercase) to the end of the constant (e.g.,0.123456789E-33L).

Character Constants



Single Character Constants

- A single character constant contains a single character, enclosed within a pair of single quote marks.

  For example: 'a', 'l', 'm', 'F', ' '(blank space)
- Character constants have integer values known as ASCII values.
- The statement
  - printf(“%d”,’a’); //print the number 97(ASCII value of a)

- Since each character constants represents an integer value ,it is also possible to perform arithmetic operations on character constants.

String constants

- A sequence of characters Enclosed in double quotes. The characters may be letters numbers, special characters and blank spaces.

  Example:

  “Hello World!”, “25”, “5”, “3+5”

Backslash character constants

-used in output functions

Character	Meaning
'\a'	alert (bell)
'\b'	backspace
'\f'	form feed
'\n'	newline
'\r'	carriage return
'\t'	horizontal tab
'\v'	vertical tab
'\\ '	backslash
'\''	single quote
'\"'	double quote
'\?'	question mark

-each one of them represents one character,although they consists of two characters.

-

these characters combinations are known as escape sequences

SYMBOLIC CONSTANTS

Symbolic constant is defined as follows

#define symbolic_name value-of-constant

Example:

```
#define PI   3.14159

#define  MAXMARKS   100
```

Rules

1)Symbolic names have the same form as variable names.

2)No blank space between the # sign define is permitted.

3)# must be the first character in the line.

4)A blank space is required between #define and symbolic name and between the symbolic name and constant.

5)#define statements must not end with a semicolon.

6)After definition, the symbolic name should not be assigned any other value within the program by using an assignment statement.

7)Symbolic names are not declared for data types. Its data type depends on the type of constant.

8)#define statements may appear anywhere in the program but before it is referenced in the program.

Variables

- In programming, a variable is a container (storage area) to hold data.
- To indicate the storage area, each variable should be given a unique name (identifier).
- Variable names are just the symbolic representation of a memory location. For example:

```
int playerScore = 95;
```

- Here, playerScore is a variable of integer type. The variable is assigned value: 95.
- The value of a variable can be changed, hence the name 'variable'.
- In C programming, you have to declare a variable before you can use it.

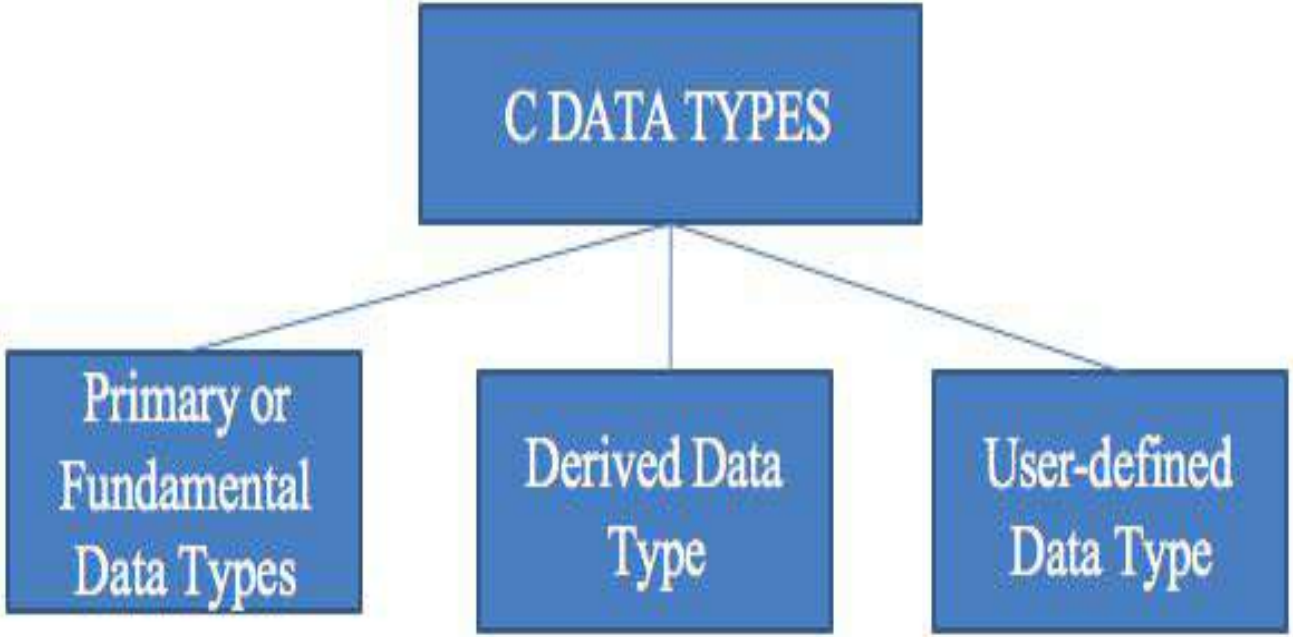
Rules for naming a variable in C

- A variable name can have letters (both uppercase and lowercase letters), digits and underscore only.
- The first letter of a variable should be either a letter or an underscore. However, it is discouraged to start variable name with an underscore.
- ANSI standard recognizes a length of 31 characters.However,length should not be normally more than eight characters,since only first eight characters are treated as significant by many compilers.
- Uppercase and lowercase are significant.i.e,the variable Total is not same as total or TOTAL
- It should not be a keyword
- White space is not allowed.

Data Types

- Data types in the c programming language are used to specify what kind of value can be stored in a variable.
- The memory size and type of the value of a variable are determined by the variable data type.
- In the c programming language, data types are classified as follows

    1. Primary (or fundamental)data types
    2. Derived data types
    3. User defined data types


C DATA TYPES
Primary or Fundamental Data Types
Derived Data Type
User-defined Data Type

PRIMARY OR FUNDAMENTAL DATA TYPES

- All C support five fundamental data types, namely

  integer (int), character (char), floating point (float), double-precision floating point (double)and void.

- Many of them also offer extended data types such as long int and long double.

PRIMARY DATA TYPES

Integral Type

Integer

signed	unsigned type
int	unsigned int
short int	unsigned short int
long int	unsigned long int

Character

char
signed char
unsigned char

Floating point Type

float	double	Long double

void

Integer datatype

- Integers are whole numbers with a range of values supported by a particular machine.
- If we use a 16 bit word length, the size of the integer value is limited to the range –32768 to +32767
- Similarly, a 32 bit word length can store an integer ranging from -2,147,483,648 to 2,147,483,647.
- In order to provide some control over the range of numbers and storage space, C has three classes of integer storage, namely short int, int, and long int, in both signed and unsigned forms.

TYPE	KEYWORD	SIZE (BITS)	RANGE
Character	char or signed char	8	-128 to 127
	unsigned char	8	0 to 255
Integer	int or signed int	16	-32,768 to 32767
	unsigned int	16	0 to 65535
	short int or signed short int	8	-128 to 127
	unsigned short int	8	0 to 255
	long int or signed long int	32	-2,147,483,648 to 2,147,483,647
	unsigned long int	32	0 to 4,294,967,295
Real	float	32	3.4E-38 to 3.4E+38
	double	64	1.7E-308 to 1.7E+308
	long double	80	3.4E-4932 to 1.1E+4932

Floating point datatype

Floating point (or real) numbers are stored in 32 bits (on all 16 bit and 32 bit machines), with 6 digits of precision.

- Floating point numbers are defined in C by the keyword ***float***.
- When the accuracy provided by a float number is not sufficient, the type double can be used to define the number.
- A double data type number uses 64 bits giving a precision of 14 digits. These are known as double precision numbers.
- Double type represents the same data type that float represents, but with a greater precision.
- To extend the precision further, we may use long double which uses 80 bits.

Character datatype

A single character can be defined as a character(char) type data.

- Characters are usually stored in 8 bits (one byte) of internal storage.
- The qualifier signed or unsigned may be explicitly applied to char. While unsigned chars have values between 0 and 255, signed chars have values from –128 to 127.

Void data type

The void type has no values.

- This is usually used to specify the type of functions.
- The type of a function is said to be void when it does not return any value to the calling function.

DECLARATION OF VARIABLES

Declaration does two things

1. It tells the compiler what the variable name is.
2. It specifies what type of data the variable will hold

Primary type declaration

A variable can be used to store a value of any data type.

Syntax for declaring a variable

data_type v1,v2,v3,….,vn;

v1,v2,v3,…vn are the name of the variables. Variables are separated by commas.

Declaration statement end with a semicolon.

Example:

```
int count;
float total;
double ratio;
```

Operators and Expressions

- An operator is a symbol that tells the computer to perform certain mathematical or logical manipulations.

- C supports a rich set of operators.

- Operators are used in programs to manipulate data and variables.

- They usually form a part of the mathematical of logical expressions.

- An ***expression*** is a sequence of operands and operators that reduces to a single value.The value can be any type other than void.

C operators are classified into a number of categories. They include:

1. Arithmetic operators
2. Relational operators
3. Logical operators
4. Assignment operators
5. Increment and Decrement operators
6. Conditional operators
7. Bitwise operators
8. Special operators

ARITHMETIC OPERATORS

Operator	Meaning
+	Addition or unary plus
-	Subtraction or unary minus
*	Multiplication
/	Division
%	Modulo division

Integer Arithmetic

When both the operands in a single arithmetic expression are integers, the expression is called an *integer expression* , and the operation is called *integer arithmetic*.

During modulo division the sign of the result is always the sign of the first operand.

That is

```
-14 % 3 = -2
-14 % -3 = -2
 14 % -3 = 2
```

Real Arithmetic

An arithmetic operation involving only real operands is called *real arithmetic*. If **x and y** are floats then we will have:

1) x = 6.0 / 7.0 = 0.857143
2) y = 1.0 / 3.0 = 0.333333

The operator % cannot be used with real operands.

Mixed-mode Arithmetic

When one of the operands is real and the other is integer, the expression is called *a mixed-mode arithmetic* expression and its result is always a real number.

Eg: 1) 15 / 10.0 = 1.5

RELATIONAL OPERATORS

- Comparisons can be done with the help of relational operators.
- The expression containing a relational operator is termed as a ***relational expression***.

  Ex : a<b , 1<20

- The value of a relational expression is either ***one*** or ***zero.***
- It is one if the specified relation is true and zero if the relation is false.
- C supports six relational operators.

Relation operators and meaning

Operator	Meaning
==	Equal
!=	Not equal
<	Less than
<=	Less than or equal
>	Greater than
>=	Greater than or equal

Syntax

ae-1 relational operator ae-2

Examples

Relational expression	Result
4.5<10	true
4.5<=10	true
4.5 ==10	false
4.5 !=10	true
4.5 >10	false
4.5>=10	false

ae-1 and ae-2 are arithmetic expressions,which may be simple constants,variables or combination of them.

Ex: 4.5<10 TRUE
10<7+5 TRUE
-35>=0 FALSE

LOGICAL OPERATORS

- These operators are used to perform logical operations on the given expressions.
- There are 3 logical operators in C language.

Logical Operators and Meaning

Operators	Meaning
&&	Logical AND
||	Logical OR
!	Logical NOT

- The logical operators && and || are used when we want to test more than one condition and make decisions.

Syntax

Op-1 logicaloperator op-2

  - Ex: a>b && x==10

Like simple relational expressions , a logical expression also yields a value of one or zero ,according to the truth table

Truth Table

Op-1	Op-2	Value of the Expression	
		Op-1&&op-2	Op-1||op-2
Non-zero	Non-zero	1	1
Non-zero	0	0	1
0	Non-zero	0	1
0	0	0	0

ASSIGNMENT OPERATORS

- Assignment operators are used to assign the result of an expression to a variables.
- The usual assignment operator is ‘ = ‘
- In addition C has a set of **shorthand assignment operators** of the form

v op=exp;

- Where ***v*** is a variable, ***exp*** is an expression and ***op*** is a binary arithmetic operator.The operator ***op***= is known as shorthand assignment operator
- The assignment statement ***v op=exp;*** is equivalent to ***v=v op (exp);***

Simple Assignment Operator	Shorthand Assignment Operator
x = x+1	x += 1
x = x-1	x -= 1
x = x*(n+1)	x *= n+1
x = x / (n+1)	x /= n+1
x = x % (n+1)	x %= n+1

Advantages of shorthand assignment operators

1. What appears on the left-hand side need not be repeated and therefore it becomes easier to write.
2. The statement is more concise and easier to read.
3. The statement is more efficient.

INCREMENT & DECREMENT OPERATORS

- Increment operators are used to increase the value of the variable by one and decrement operators are used to decrease the value of the variable by one in C programs.

- Syntax:

  Increment operator: ++var_name; (or) var_name++;

  Decrement operator: – -var_name; (or) var_name – -;

- Example:

  Increment operator :++ i ; i ++ ;

  Decrement operator : – – i ; i – – ;

➢Prefix operator adds 1 to the operand and then the result is assigned to the variable on left.

➢Postfix operator first assigns the value to the variable on left and then increments the operator.

Examples : suppose m=5;

operator	Expression	Result	M value
Prefix ++	y=++m	y=6	m=6
Prefix --	y=--m	y=4	m=4
Postfix ++	y=m++	y=5	m=6
Postfix --	y=m--	y=5	m=4

Rules for ++ and - - operator

1.Increment and decrement operators are unary operators.

2.When postfix ++ (or --) is used with a variable in an expression, the expression is evaluated first using the original value of the variable and then the variable is Increment (or decrement) by one.

3.When prefix ++ (or decrement) is used in an expression, the variable is incremented (or decremented) first and then the expression is evaluated using the new value of the variable.

4.The precedence and associativity of ++ and – operators are same as those of unary + and unary -.

CONDITIONAL OPERATOR

- Conditional operator contains a condition followed by two statements or values.If the condition is true,the first statement is executed otherwise the second statement .

- The conditional operator ? : are sometimes called ternary operators because they take three arguments.

Syntax

exp1? (exp2):(exp3);

Where exp1, exp2 and exp3 are expression.

➢Exp1 is evaluated first.

➢If it is true(nonzero) then the expression exp2 is evaluated and becomes the value of the expression.

➢Otherwise exp3 is evaluated and its value becomes the value of the expression.

Example

a=20; b=15;

x=(a>b)?a:b;

output

x=20

BITWISE OPERATORS

Bitwise operators

➢Bitwise operators are used for manipulating data at bit level.

➢These operators are used for testing the bits, or shifting them right or left.

➢Bitwise operators may not be applied to float or double.

Operators	Meaning
&	Bitwise AND
|	Bitwise OR
^	Bitwise exclusive OR
<<	Shift left
>>	Shift right

SPECIAL OPERATORS

Special operators are

1. comma operator (,)
2. sizeof operator
3. pointer operators (& and *)
4. member selection operators(.,->)

Comma operator

- Comma operator can be used to link the related expression together.
- A comma-linked list of expressions are evaluated left to right and the value of right-most expression is the value of the combined expression.

Example:

z=(x=10, y=15, x+y);

➢First assigns the value 10 to x, then 15 to y and finally 25 to z.

➢Comma operator has the lowest precedence of all operators.

sizeof operator

- The sizeof is a compile time operator and when used with an operand, it returns the number of bytes the operand occupies.
- The operand may be a variable, a constant or a data type qualifier.

Examples:

```
m=sizeof(sum);

n=sizeof(long int);

k=sizeof(235l);
```

Arithmetic Expressions

An **expression is a combination of variables constants and operators** written according to the syntax of C language.

In C every expression **evaluates to a value i.e., every expression results in some value of a certain type that can be assigned to a variable.**

Some examples of C expressions are shown in the table given below.

- **<u>Evaluation of Expressions</u>**

Expressions are evaluated using an assignment statement of the form

- **Variable = expression;**
- Variable is any valid C variable name. When the statement is encountered, the expression is evaluated first and then replaces the previous value of the variable on the left hand side. All variables used in the expression must be assigned values before evaluation is attempted.

Example of evaluation statements are

- x = a * b – c
- y = b / c * a
- z = a – b / c + d;

Precedence in Arithmetic Operators

- An arithmetic expression without parenthesis will be evaluated from left to right using the rules of precedence of operators. There are two distinct priority levels of arithmetic operators in C.

High priority * / %
Low priority + -

```
main()
{
int a = 20,b = 10, c = 15,d = 5;
int e;
e = (a + b) * c / d;     // ( 30 * 15 ) / 5
printf("Value of (a + b) * c / d is :%d\n", e );
e = ((a + b) * c) / d;     // (30 * 15 ) / 5
printf("Value of ((a + b) * c) / d is :%d\n" , e );
e = (a + b) * (c / d); // (30) * (15/5)
printf("Value of (a + b) * (c / d) is :%d\n", e );
e = a + (b * c) / d; // 20 + (150/5)
printf("Value of a + (b * c) / d is :%d\n" , e );
}
```

Output:

Value of (a + b) * c / d is :90

Value of ((a + b) * c) / d is :90

Value of (a + b) * (c / d) is :90

Value of a + (b * c) / d is :50

Rules for evaluation of expression

- 1. **First parenthesized sub expression** left to right are **evaluated.**
  2. **If parentheses is nested**, the evaluation **begins with the innermost sub expression.**
  3. **The precedence rule is applied** in determining the **order of** application of **operators** in **evaluating sub expressions.**
  4. The associability rule is applied when two or more operators of the same precedence level appear in the sub expression.
  5. **Arithmetic expressions are evaluated from left to right** using the rules of precedence.
  6. When Parentheses are used, the **expressions within parenthesis** assume **highest priority**

Operator precedence and associativity

- **Each operator in C has a precedence associated with it**.
- **The precedence is used to determine how an expression involving more than one operator is evaluated.** There are distinct levels of precedence and an operator may belong to one of these levels.
- **The operators of higher precedence are evaluated first.**
- **The operators of same precedence are evaluated from right to left or from left to right depending on the level. This is known as associativity property of an operator.**

Operator	Description	Precedence level	Associativity
( ) [ ] . ->> ++ --	Parentheses: grouping or function call Brackets (array subscript) Dot operator (Member selection via object name) Arrow operator(Member selection via pointer) Postfix increment/decrement	1	Left to Right
+ - ++ -- ! ~ * & (datatype) sizeof	Unary plus Unary minus Prefix increment/decrement Logical NOT One's complement Indirection Address (of operand) Type cast Determine size in bytes on this implementation	2	Right to Left
* / %	Multiplication Division Modulus	3	Left to Right
+ -	Addition Subtraction	4	Left to Right
<< >>	Left shift Right shift	5	Left to Right
< <= > >=	Less than Less than or equal to Greater than Greater than or equal to	6	Left to Right
== !=	Equal to Not equal to	7	Left to Right
&	Bitwise AND	8	Left to Right
^	Bitwise XOR	9	Left to Right
|	Bitwise OR	10	Left to Right
&&	Logical AND	11	Left to Right
||	Logical OR	12	Left to Right
? :	Conditional operator	13	Right to Left
= *= /= %= += -= &= ^= |= <<= >>=	Assignment operators	14	Right to Left
,	Comma operator	15	Left to Right

Structure of a C program

A C program is divided into different sections. There are six main sections to a basic c program.

Documentation Section

The documentation section is the part of the program where the programmer gives the details associated with the program. He usually gives the name of the program, the details of the author and other details like the time of coding and description. It gives anyone reading the code the overview of the code.

```
/* File Name: areaofcircle.c

Author: Abhijith

date: 06/05/2021

 description: a program to calculate area of circle

user enters the3 radius6

*/
```

Link Section

This part of the code is used to declare all the header files that will be used in the program. This leads to the compiler being told to include the header files to the source program and link the system libraries.

```
#include<stdio.h>
```

Definition Section

In this section, we define different constants. The keyword define is used in this part.

#define PI 3.14

Global Declaration Section

This part of the code is the part where the global variables are declared. All the global variable used are declared in this part. The user-defined functions are also declared in this part of the code.

```
float area(float r);

int a=7;
```

Main Function Section

Every C-programs needs to have the main function.

Each main function contains 2 parts.

A declaration part and an Execution part.

The declaration part is the part where all the variables are declared. The execution part begins with the curly brackets and ends with the curly close bracket. Both the declaration and execution part are inside the curly braces.

```
int main()//main function
{
float r;
printf(" Enter the radius:\n");
scanf("%f",&r);
printf("the area is: %f",area(r));
return 0;
}
```

Sub Program Section

All the user-defined functions are defined in this section of the program.

```
float area(float r)

{

return pi * r * r;//sub program

}
```

DECISION MAKING AND BRANCHING

Branching:

- The C language programs presented until now follows a sequential form of execution of statements. Many times it is required to alter the flow of the sequence of instructions.
- **C language provides statements that can alter the flow of a sequence of instructions**. These statements are called **Control statements or Decision-making statements**..
- These statements **help to jump from one part of the program to another.**
- The **control transfer may be conditional or unconditional.**
- Decision making is used to see whether a particular condition has occurred or not and then direct the computer to execute certain statements accordingly.

Decision making statements are used to execute a part of statement based on some condition and exclude other.

- **C language possesses such decision-making** capabilities by supporting the following statements:-

1. **If statement**
2. **Switch statement**
3. **conditional operator statement**
4. **goto statement**

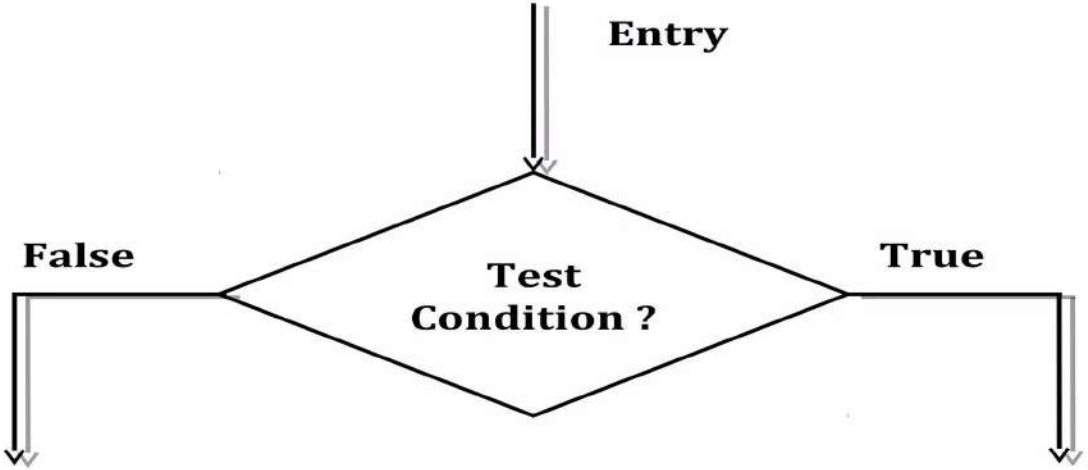
Decision making with if Statement:

- **The if statement** is a powerful **decision-making statement** and is used **to control the flow of execution of statements**.
- It is basically a **two-way decision statement** and is **used in conjunction with an expression.**

The if structure has the following syntax

- **if (condition)**
  statement;
- It allows the computer to **evaluate the expression first** and then depending on **whether the value of the expression is true** (or non-zero) or 'false' (zero), it **transfers the control to a particular statement.**

IF Statement



Example:

```
If(Pocket balance is zero)
        Borrow money;
```

IF Statement

- The if statement can be implemented if four different forms depending on the complexity of the conditions to be tested. The four forms are:

1. Simple if statement
2. If else statement
3. Nested if else statement
4. Else if ladder

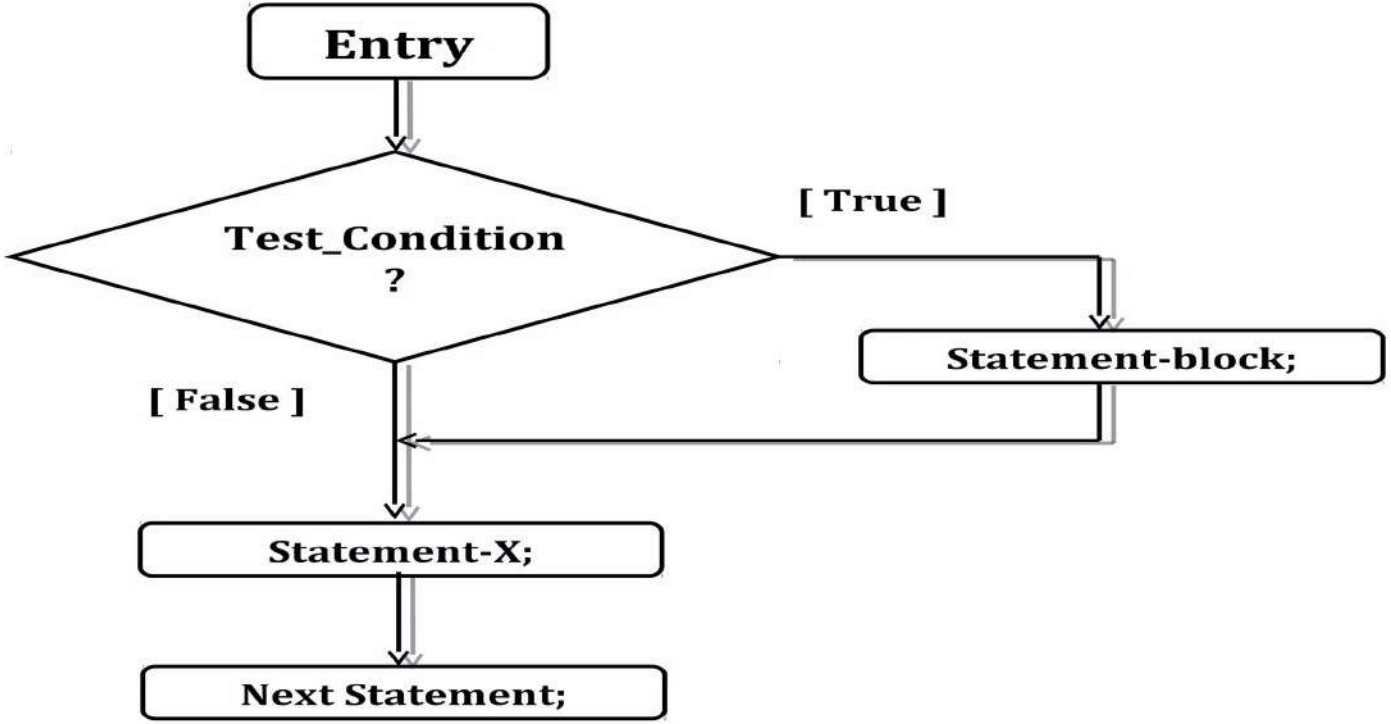
Simple if statement:-

The general form of a **simple if** statement is as follows,

```
if (test expression)
{
    statement-block;
}
statement – x;
```

- Here, the statement block may contain a single statement or a group of statements.
- If the test expression is **true** then the statement block will be executed; otherwise it will be skipped to the statement – x.

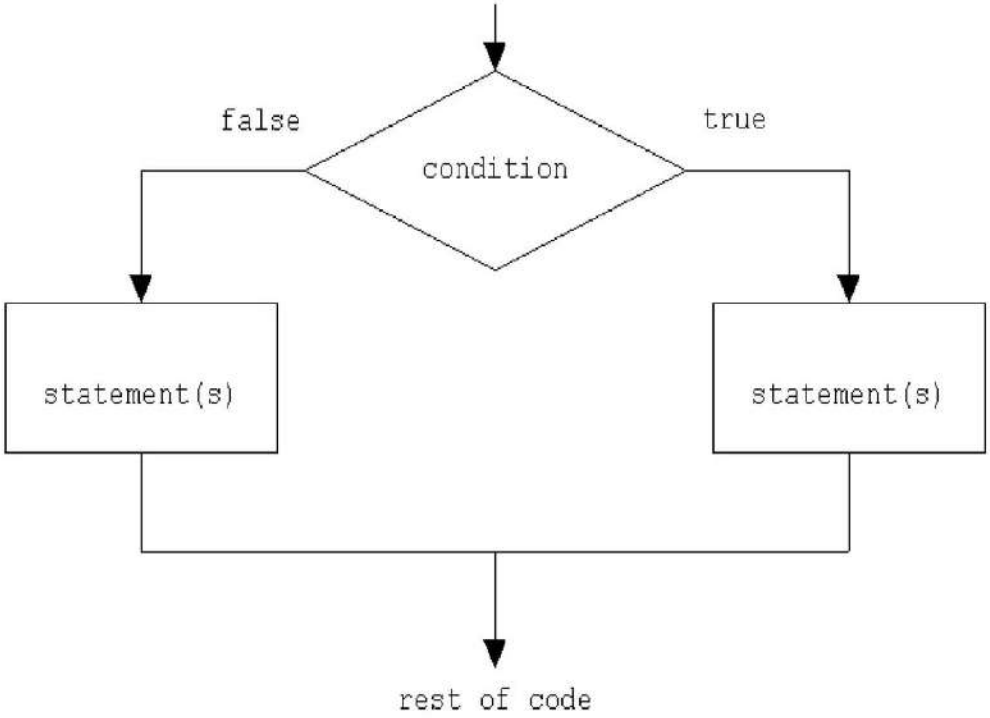
Simple if Statement- Flowchart



```
#include<stdio.h>
void main()
{
    int number;
    printf("Type a number:");
    scanf("%d", &number);
    if (number < 0)
    {
        number = -number;
   }
        printf("The absolute value is %d\n", number);
}
```

- The above program **checks the value of the input number to see if it is less than zero.**
- If it is then the following **program statement which negates the value of the number is executed.**
- If the value of the number is not less than zero, we do not want to negate it then this statement is automatically skipped.
- The **absolute number is then displayed** by the program, and program execution ends.

- **<u>The if else construct:</u>**
- The syntax of the ***If else construct*** is as follows:-




```
if (test expression)
   statement1;
else
   statement2;
```

if else Statement- Structure

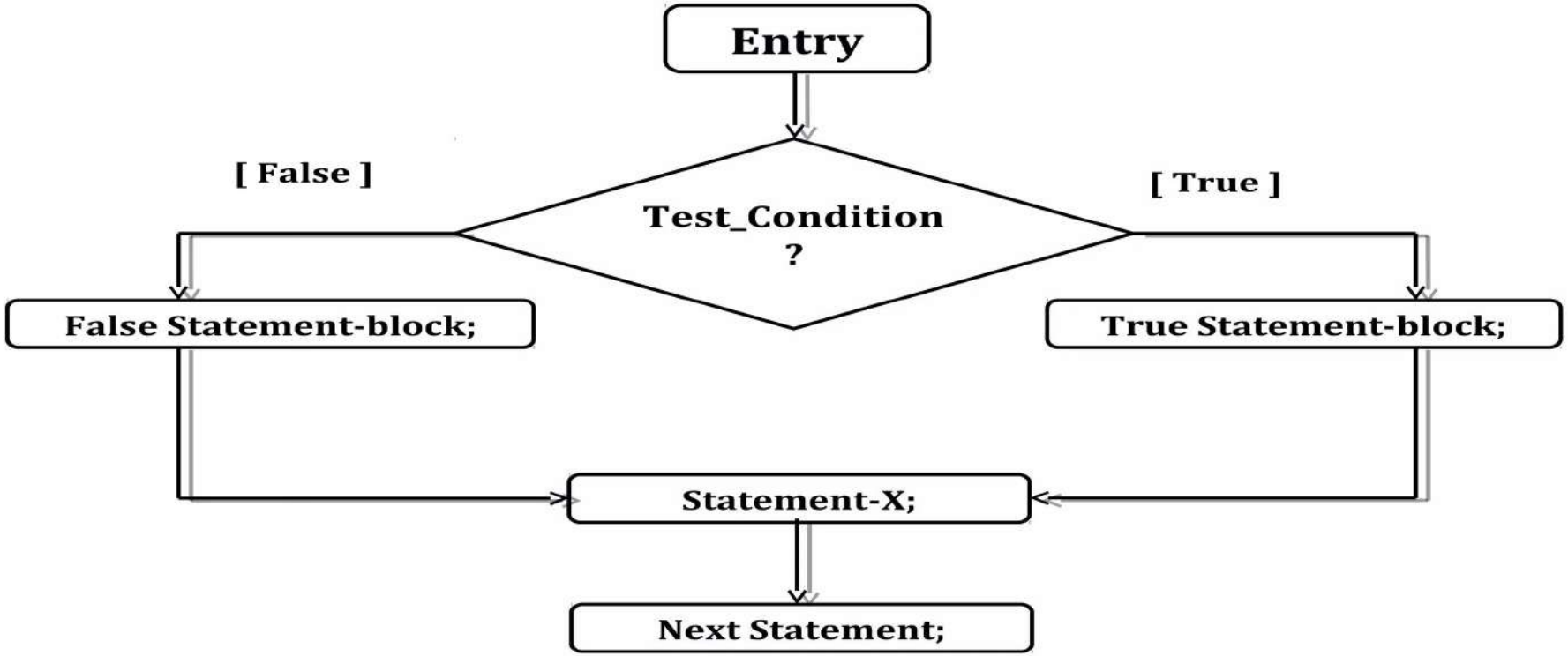
- The if else statement is an extension of the simple if statement. The general form is :

```
if (test_condition)
      {
      True block statements;
      }
else
      {
      False block statements;
      }
statement-x;
```

if else Statement

- If the test condition is true then the true block statements, immediately following the if statements are executed;
- Otherwise the false block statements are executed.
- In short either true-block or false-block of statements will be executed, not both.
- But in both cases the control is transferred subsequently to the statement-x as it is an independent (not controlled by the if else statement) statement.
- **It is also called two way conditional branching**

if else Statement- Flowchart



The following program checks whether the entered number is positive or negative.

```
#include<stdio.h>
int main( )
{
int a;
printf("n Enter a number:");
scanf("%d",&a);
if(a>0)
{
printf("n The number %d is positive.",a);
}
else
{
printf("The number %d is negative.",a);
}
return 0;
}
```

Nested if Statement

When a series of decisions are involved, we may have to use more than one if..else statement in nested form.

The if statement may itself contain another if statement is known as nested if statement.

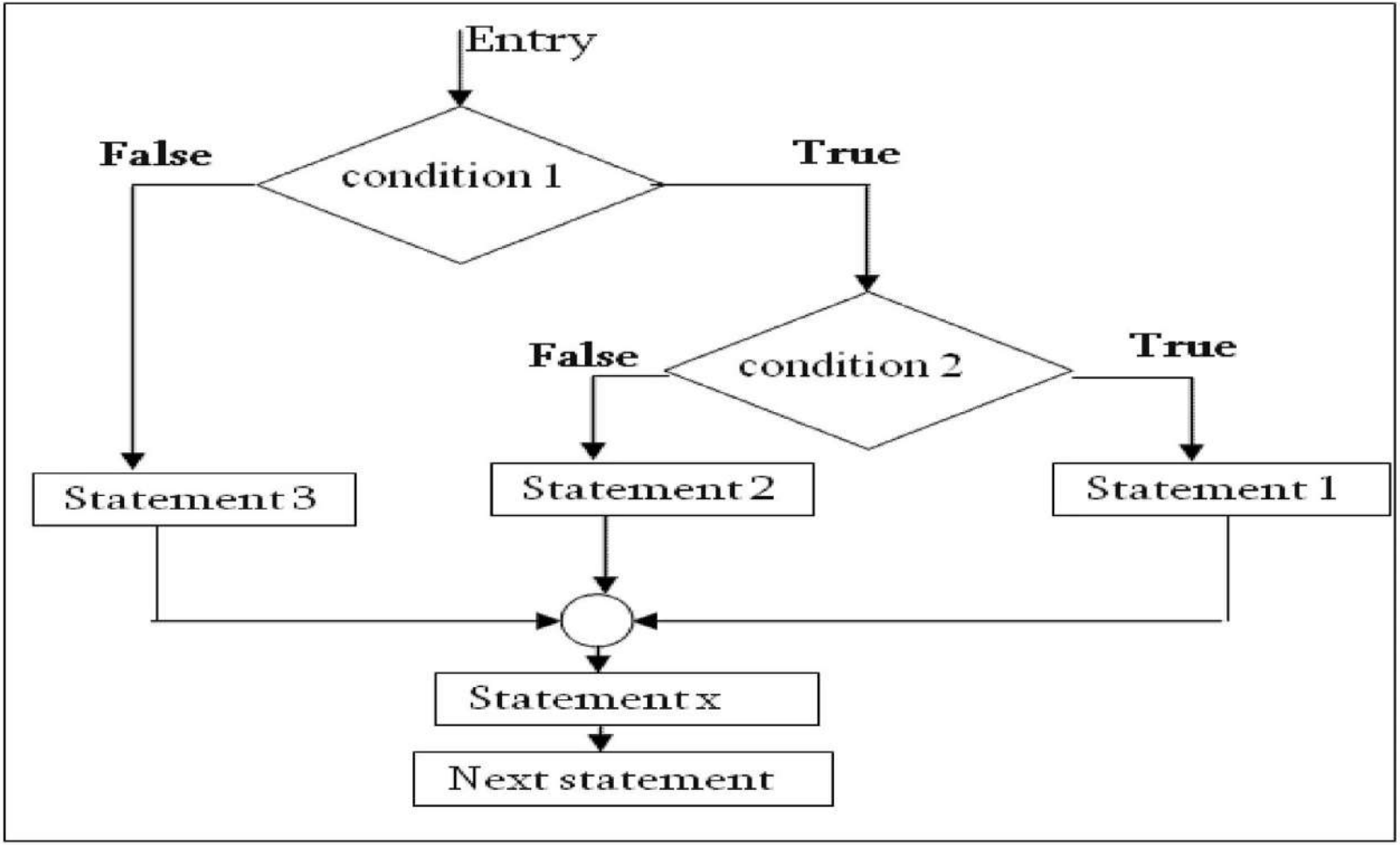
Syntax:

Nested if else Statement- Structure

```
if (test_condition)
    {
        if (test_condition)
            {
                statement-block;
            }
        else
            {
                statement-block;
            }
    }
else
    {
        statement-block;
    }
statement-x;
```

Nested if else

- During the execution of a nested if-else statement, as soon as a condition is encountered which evaluates to true, the statements associated with that particular if-block will be executed and the remainder of the nested if-else statements will be bypassed.
- If neither of the conditions are true, either the last else-block is executed or if the else-block is absent, the control gets transferred to the next instruction present immediately after the else-if ladder.


Entry
False
condition 1
True
False
condition 2
True
Statement 3
Statement 2
Statement 1
Statement x
Next statement

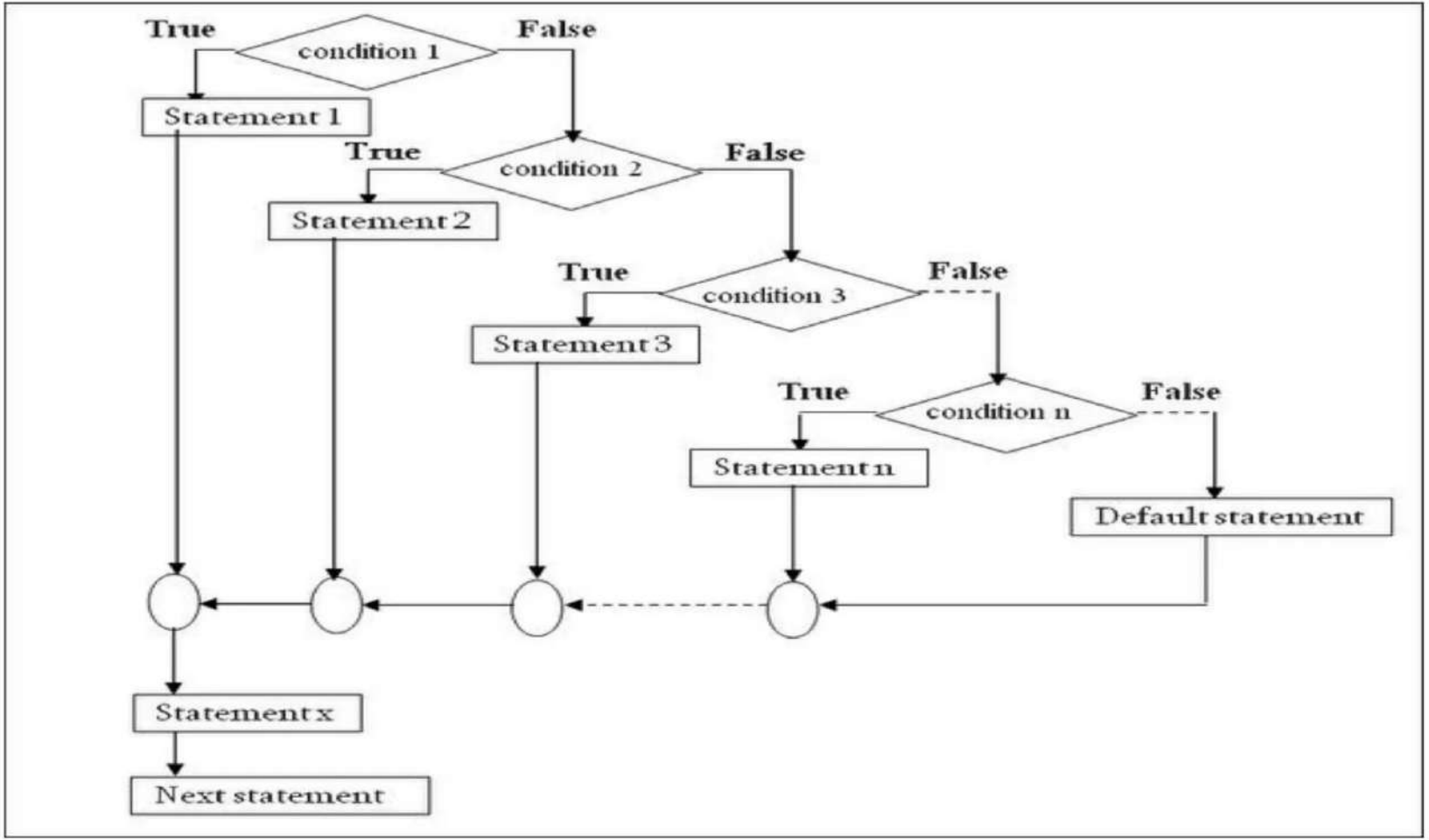
```
#include <stdio.h>

void  main() {

    int num = 10;

    if (num > 0) {

        printf("Number is positive.\n");

          if (num % 2 == 0) {

            printf("Number is even.\n");

        } else {

            printf("Number is odd.\n");

        }

    }

 else

 {  printf("Number is non-positive.\n");

    }

}
```

The ELSE If Ladder:

- The ifs are put together when multipath decisions are involved. A multipath decision is a chain of ifs in which that statement associated with each else is an if.

Syntax:

```
• if (condition1)
        statement1;
  else if (condition2)
         statement2;
  else if (condition3)
        statement3;
  .....
 else
        default statement;
statement-x;
```


True
condition 1
False
Statement 1
True
condition 2
False
Statement 2
True
condition 3
False
Statement 3
True
condition n
False
Statement n
Default statement
Statement x
Next statement

```c
#include <stdio.h>
void main() {
  int num1, num2, num3;
  printf("Enter three numbers: ");
  scanf("%d %d %d", &num1,&num2,&num3);
  if (num1 >= num2 && num1 >= num3) {
     printf("%d is the largest number.\n", num1);
  }
  else if (num2 >= num1 && num2 >= num3) {
     printf("%d is the largest number.\n", num2);
  }
  else {
     printf("%d is the largest number.\n", num3);
  }
}
```

The Switch Statement:

- Unlike the **If statement which allows a selection of two alternatives** the **switch statement allows a program to select one statement for execution out of a set of alternatives.**
- During the execution of the switch statement **only one of the possible statements will be executed** the remaining statements will be skipped.
- The usage of multiple **If else statement increases the complexity of the program** since when the number of If else statements increase it affects the readability of the program and makes it difficult to follow the program.
- The switch statement removes these disadvantages by using a simple and straight forward approach

```
switch (expression)
{
case value1: block-1
                 break;

case value2: block-2
                 break;
.............
default:       default-block
                break;
}
statement-x;
```

- The expression is an integer expression or characters.

- Value-1, value-2... are constants or constant expressions and are known as case labels.

- Each of these values should be unique within a switch statement.

- block-1, block-2...are statement list and may contains zero or more statements.

- There is no need to put braces around these blocks.
- case labels end with a colon (:).

- When the switch is executed, the value of the expression is successfully compared against the values value-1, value-2,...

- if a case is found whose value matches with the value of the expression, then the block of statements that follows the case are executed.

- The break statement at the end of each block signal the end of a particular case and causes an exit from the switch statement, transferring the control to the statement-x following the switch.

- The default is an optional case.

- When present, it will be executed if the value of the expression does not match with any of the case values.

- If not present, no action takes place if all matches fail and the control goes to the statement-x.

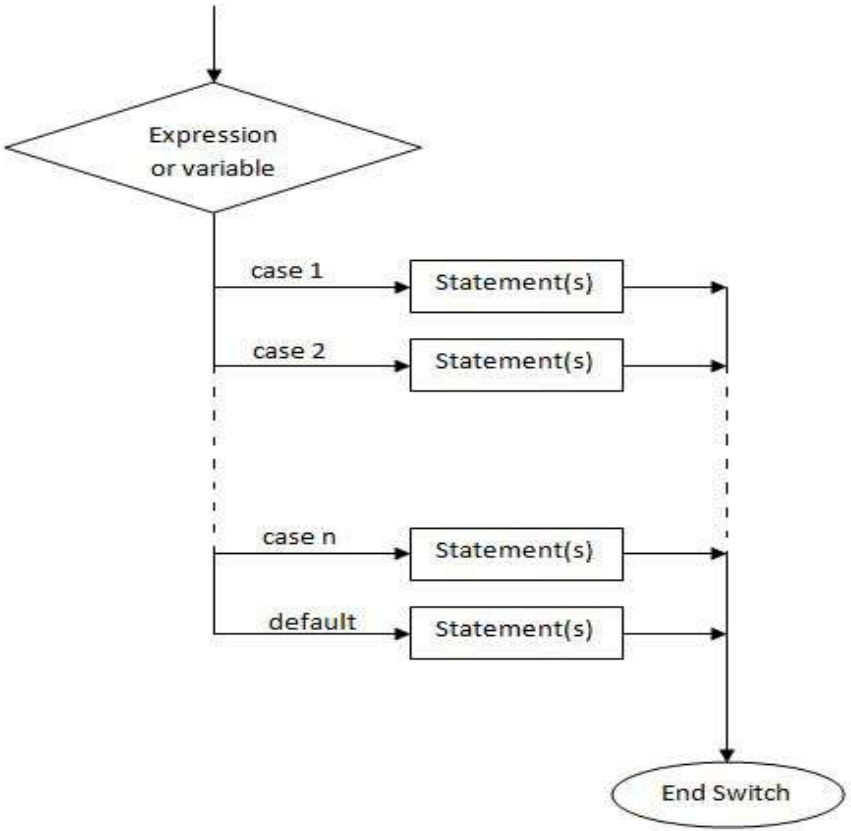


fig: Flowchart for switch case statement

Rules for switch statement

The switch expression must be an integral type.

Case labels must be constants or constant expressions.

Case labels must be unique. No two labels can have the same value.

Case labels must end with semicolon.

The default label is optional. If present, it will be executed when the expression does not find a matching case labels.

There can be at most one default label.

The default may be placed anywhere but usually placed at the end.

It is permitted to nest switch statements.

```
/*Program to find whether the number is odd or even*/
#include <stdio.h>

main()
{
   int n;
   printf("Enter the number");
   scanf("%d",&n);

      switch(n%2)
      {
         case 0 : printf("\nThe number %d is even",n);
                  break;
         case 1 : printf("\nThe number %d is odd \n",n);
                  break;
      }
 }
```

The ?: operator:-

- The conditional operator or ternary operator **is useful for making two-way decisions**. This operator is a combination of ? and :, and takes 3 operands.
- The general form of use of the conditional operator is as follows:

Conditional expression ? expression1 : expression2

Example:

flag = (x < 0) ? 0 : 1;

- The conditional expression is evaluated first. If the result is nonzero, expression1 is evaluated and is returned as the value of the conditional expression. Otherwise, expression2 is evaluated and its value is returned.

For example the statement

```
if (x <0)
        flag=0;
else
        flag=1;
```

can be written as `flag = (x<0) ? 0 : 1;`

Example:

```
#include <stdio.h>
#include <conio.h>
#include <math.h>
void main()  //If True first exp is executed
{
   int a;    //If False second exp is executed
   printf("Enter a number: ");
   scanf("%d",&a);
  (a%2==0?printf("Even"):printf("Odd"));
  getch();
}
```

```
Enter a number: 9
Odd
```

```
Enter a number: 8
Even
```

The goto statement:-

- C supports the goto statement **to branch unconditionally form one point to another in the program.**

goto label;

- The goto statement is simple statement **used to transfer the program control unconditionally from one statement to another statement**.

Eg:

- **goto read;**

```
goto label;
.............
.............
.............
Label:
Statement;
```

```
label:
.............
.............
.............
goto label;
```

The goto requires a label in order to identify the place where the branch is to be made.

A label is a valid variable name followed by a colon.

DECISION MAKING AND LOOPING

Iteration Statements

- Iteration statements is also known as Looping statement.
- A segment of the program that is executed repeatedly is called as a loop.
- Some portion of the program has to be specified several number of times or until a particular condition is satisfied.
- Such repetitive operation is done through a loop structure.
- The Three methods by which you can repeat a part of a program are,
  - **while Loops**
  - **do….while loops**
  - **for Loop**

Loops generally consist of two parts :

Control expressions: One or more *control expressions* which control the execution of the loop,

Body : which is the statement or set of statements which is executed over and over

Any looping statement , would include the following steps:

- Initialization of a condition variable
- Test the control statement.
- Executing the body of the loop depending on the condition.
- Updating the condition variable.

- Depending on the position of the control statement in the loop, a control structure can be classified into two types;

1. **entry controlled and**
2. **exit controlled.**

- **<u>Entry controlled loop :</u>** The types of loop where the **test condition is stated before the body of the loop,** are known as the entry controlled loop.

Entry controlled loop, the **condition is tested before the execution of the loop.** If the test condition is true, then the loop gets the execution, otherwise not.

For example, the **for loop** is an entry controlled loop.

Exit controlled loop :

- The types of loop where the **test condition is stated at the end of the body of the loop**, are know as the exit controlled loops.
- So, in the case of the exit controlled loops, the body of the loop gets executed without testing the given condition for the first time.
- Then the condition is tested.
- If it comes true, then the loop gets another execution and continues till the result of the test condition is not false.
- For example, the **do statement** or the **do....while loop** is an exit controlled loop.

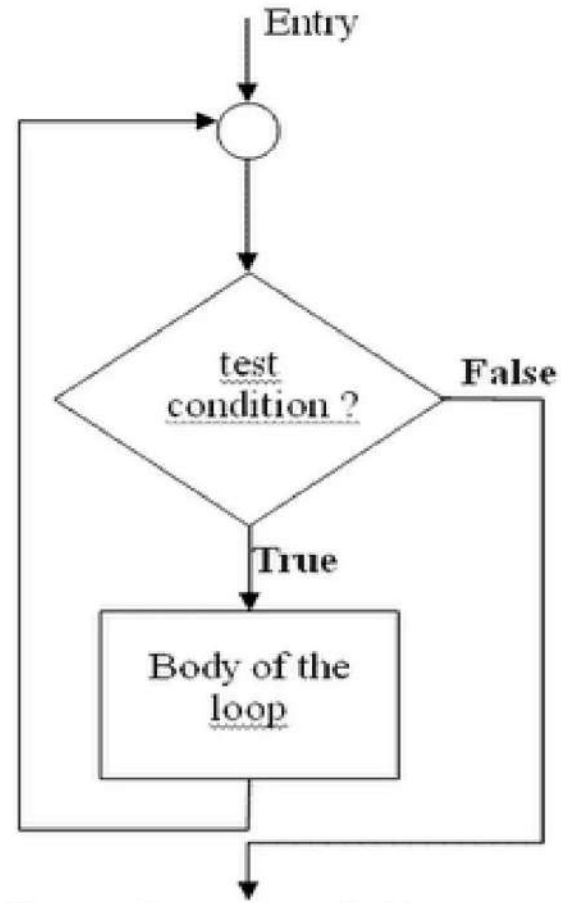


Figure : Entry controlled loop

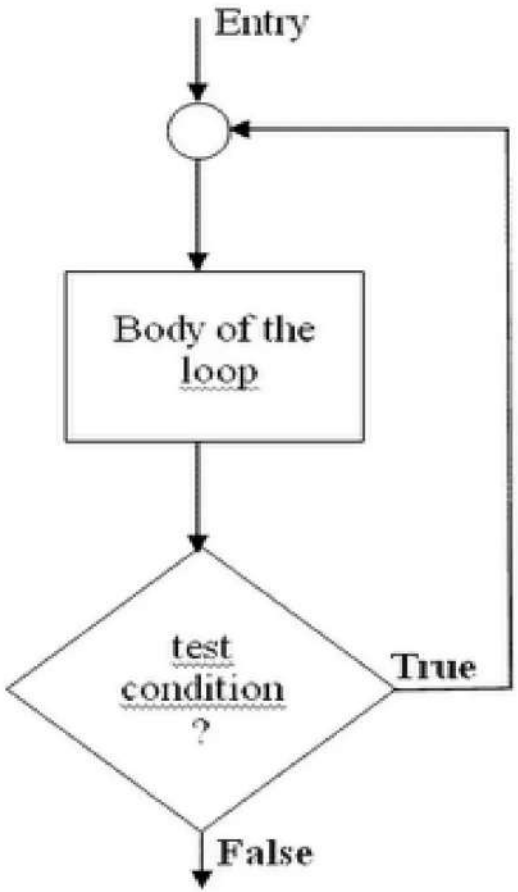


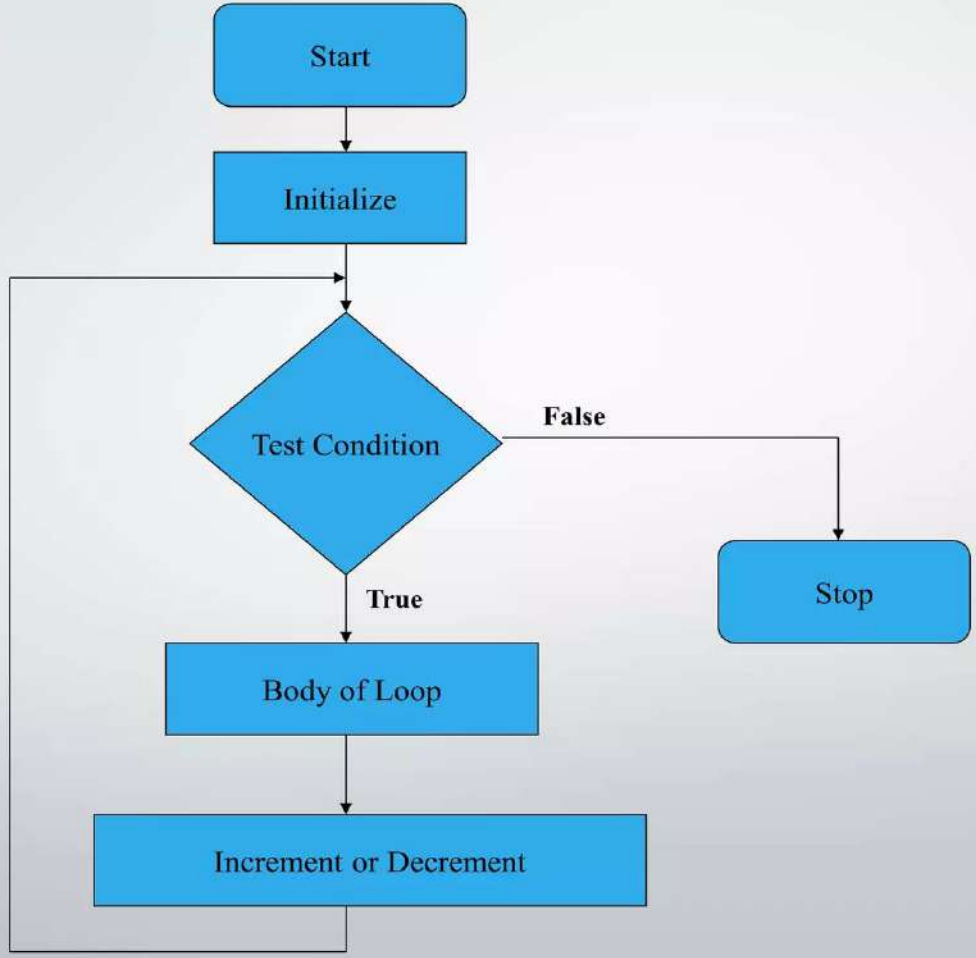
Figure : Exit controlled loop

The While Statement:

- The simplest of all looping structure in C is the while statement.
- The **general format of the while statement is:**

```
while (test condition)
{
        body of the loop
}
```

The given test condition is evaluated and if the condition is true then the body of the loop is executed. After the execution of the body, the test condition is once again evaluated and if it is true, the body is executed once again. This process of repeated execution of the body continues until the test condition finally becomes false and the control is transferred out of the loop. On exit, the program continues with the statements immediately after the body of the loop. The body of the loop may have one or more statements. The braces are needed only if the body contained two are more statements


Start
Initialize
Test Condition
False
Stop
True
Body of Loop
Increment or Decrement

Example:

```
// Print the I Values
#include <stdio.h>
void main()
{
        int i;
        i = 0;
        while(i<=10)
        {
        printf("The I Value is :%d\n", i );
            ++i;
        }
}
```

```
// Summation of the series 1 + 2 + 3 + 4 +
.....
#include <stdio.h>
void main()
{
        int i, sum;
        i = 1;
        sum = 0;
        while(i<=10)
        {
            sum = sum + I;
            printf("The Sum Value is:%d\n",i);
              ++i;
        }
}
```

Program to print the summation of digits of any given num

```
#include<stdio.h>
void main()
{
   int number=0, rem=0, sum=0;
   printf("Enter the value for number");
   scanf("%d", &n);
   while(number > 0)
   {
     rem = number % 10;
     sum = sum + rem;
     number = number / 10;
   }
   printf("the summation value of  the given number %d is = %d", number, sum);
}
```

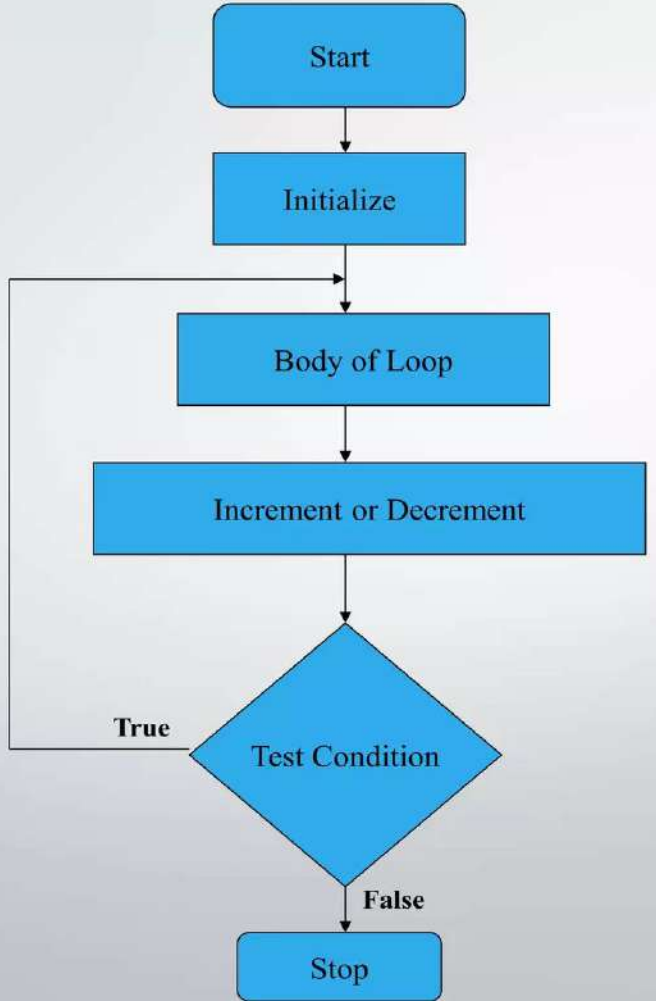
THE do-while LOOP

- The body of the loop may not be executed if the condition is not satisfied in while loop.
- Since the test is done at the end of the loop, the statements in the braces will always be executed at least once.
- The statements in the braces are executed repeatedly as long as the expression in the parentheses is true.

```
initialize loop counter;
do
{
        statement (s);
        increment or decrement loop counter
}
while (condition);
```

Make a note that do while ends in a ; (semicolon)

Note that Do… While Looping statement is Exit Controlled Looping statement


Start
Initialize
Body of Loop
Increment or Decrement
True
Test Condition
False
Stop

Difference Between While Loop and Do – While Loop

S. No.	while loop	do-while loop
1.	The while loop tests the condition before each iteration.	The do – while loop tests the condition after the first iteration.
2.	If the condition fails initially the loop is Skipped entirely even in the first iteration.	Even if the condition fails initially the loop is executed once.

Example:

```
// Print the I Values
#include <stdio.h>
void main()
{
        int i;
        i = 0;
        do
        {
            printf("The I Value is :%d\n", i);
            ++i;
        }
         while(i<=10);
}
```

```
// Print the I Values
#include <stdio.h>
void main()
{
        int i;
        i = 11;
        do
        {
            printf("The I Value is :%d\n", i);
            ++i;
        }
         while(i<=10);
}
```

Program to print the Fibonacci series for any given number Using Do....While Loop

```
#include <stdio.h>
void main()
{
   int i, f1,f2,f3;
   f1 = 0;
   f2 = 1;
   printf("The Fibonacci Series is:\n")
   printf("%d\n",f1);
   printf("%d\n",f2);
   do
   {
     f3 = f1 + f2;
     printf(%d\n", f3);
     f1 = f2;
     f2 = f3;
     ++i;
   }
   while(i <= 10);
}
```

for Loop

- The for loop is another repetitive control structure, and is used to execute set of instruction repeatedly until the condition becomes false.
- To set up an initial condition and then modify some value to perform each succeeding loop as long as some condition is true.

The syntax of a for loop is

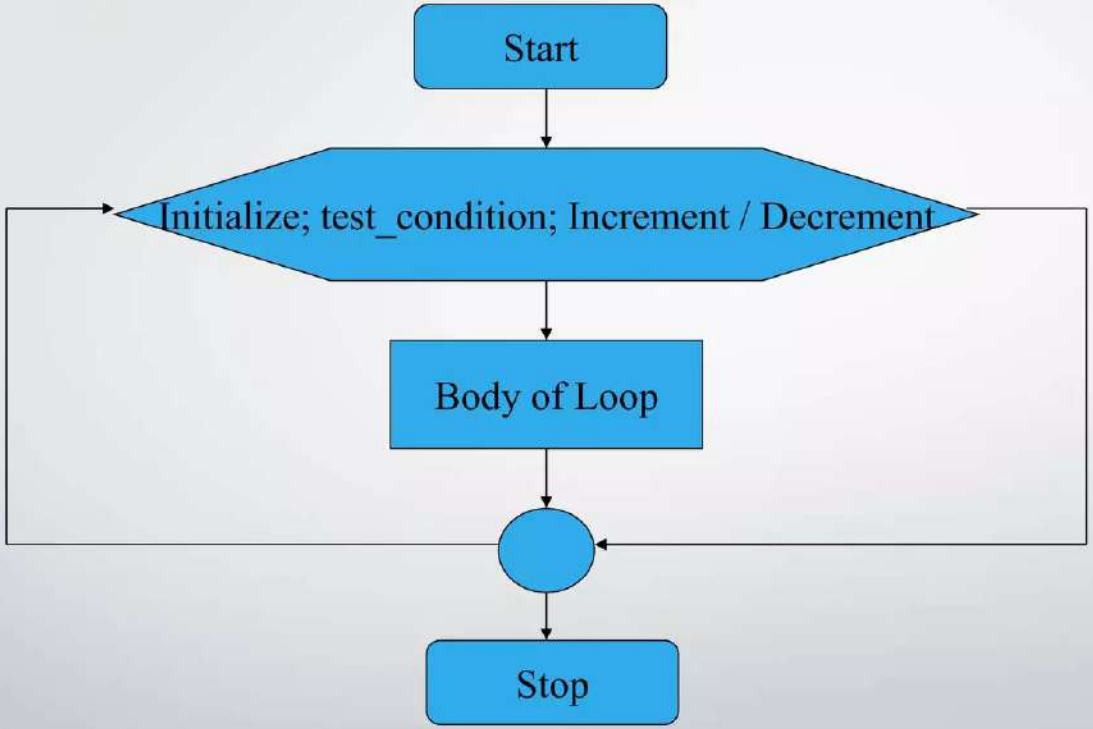
```
for( expr1; expr2 ;expr3)
{
 Body of the loop;
}
```

The three expressions :

expr1 - sets up the initial condition,

expr2 - tests whether another trip through the loop should be taken,

expr3 - increments or updates things after each trip.


Start
Initialize; test_condition; Increment / Decrement
Body of Loop
Stop

Example

Given example will print the values from 1 to 10.

```
#include<stdio.h>
void main()
{
   for (int i = 1; i <= 10; i++)
   printf("i is %d\n", i);
}
```

There is no need of { } braces for single line statement and for multiple line it is

essential else it will consider only next line of for statement.

Example

Given example of Multiplication Table

```
#include<stdio.h>
#include<conio.h>
void main()
{
   int mul, limit, c, i;
   printf("Enter the Multiplication Number:\n");
   scanf("%d", &mul);
   printf("Enter the Limits:\n");
   scanf("%d", &limit);
     for(i=1;i<=limit;i++)
   {
     c = i * mul;
     printf("%d * %d: %d\n", i, mul, c);
   }
}
```